

ADR-011 — Arquitetura de componentes cross-platform (recipes headless + adaptadores finos)

Status: Accepted (a ratificar) · **Architectural Baseline** · 2026-07-21 · **Ref:** DS-002 · ADR-007 · HIP-013

Contexto

O Design System é **único** (Web + Mobile) e os componentes devem ser compartilhados. Web = Next.js/React (DOM); Mobile = React Native + Expo. Falta definir **como** compartilhar os componentes sem violar decisões congeladas.

Decisão

Componentes fundamentais são expressos como **recipes (funções puras, headless)** no pacote `@sintera/design-system: recipe(theme, props/estado) → VisualSpec` (cores, espaçamentos, raios, tipografia, elevação, alvo de toque, intenção de acessibilidade) — **derivado 100% dos papéis do tema**, sem `react-dom`, sem `react-native`, sem hex cru. Cada plataforma tem um **adaptador fino** que mapeia o `VisualSpec` para o seu sistema de estilo (web: CSS/estilo inline; RN: `StyleSheet`). A lógica (variantes, estados, tamanhos, contraste, alvo de toque, papéis ARIA/acessibilidade) mora **uma vez** na recipe.

Alternativas consideradas

- **react-native-web** (escrever componentes RN e rodar na web): rejeitada — alteraria o **build de produção da web** (Babel/Next/rnw) e o stack Tailwind existente; contraria "não modificar decisões/produção congeladas".
- **Duas implementações completas** (React web + RN): rejeitada — duplicação, diverge com o tempo, contraria o "DS único".
- **Recipes headless + adaptadores** (escolhida): máximo reúso da lógica de design, **não toca o build da web**, totalmente verificável por `tsc` /testes de contrato no ambiente atual, e mantém uma só fonte de verdade visual.

Princípios (ratificados pela fundadora, 2026-07-21)

1. **A recipe é a fonte da verdade.** Ela representa o comportamento e a linguagem visual do componente. **Não pertence à Web nem ao Mobile — pertence ao Design System.**
2. **Adaptadores são descartáveis e puramente técnicos.** Só traduzem a recipe para a tecnologia da plataforma. **Não podem conter** regras de negócio · decisões visuais · lógica de composição · tokens.

3. **Fluxo obrigatório:** Design Tokens → Recipes headless → Adapters → (React Web | React Native) → Templates . Nenhuma implementação específica de Web ou Mobile contém regras próprias de design.
4. **Critério para todo novo componente:** (a) existe um padrão recorrente? (b) ele pode ser descrito por uma receita? (c) só então criar os adaptadores. **Nunca começar pela implementação da interface.**
5. **Componentes de domínio são componentes de PRODUTO**, não de interface: carregam o comportamento do domínio (números tabulares, referências, tendências, destaque de alterações, acessibilidade), sempre de forma **factual** (RDC 657 — sem interpretação clínica).

Consequências

O pacote permanece **framework-agnóstico e testável** (contratos ARCH validam contraste/alvo de toque/mapeamento de token). Os adaptadores de plataforma são finos e entram na fiação de cada app (web agora; RN no ambiente de desenvolvimento). A migração da web para o DS-002 consome as mesmas receitas. Governa a Etapa 3 do DS-002.